Principles of data science lab experminet:

Experiment 1:

ATTANA S

241801030 Experiment 1 – Data Preprocessing and Cleaning

[3]

```python
# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder

# 2. Load dataset
import pandas as pd

url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print(df.head())
print("Shape:", df.shape)

# 3. Display dataset
print(df.head(10))
print(df.tail(10))
print("Shape:", df.shape)
```

mands + Code + Text ▷ Run all Copy to Drive

[3]

```python
print(df.tail(10))
print("Shape:", df.shape)
print(df.info())
print(df.describe())

# 4. Check missing values and duplicates
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())

# 5. Remove duplicates
df.drop_duplicates(inplace=True)

# 6. Convert TotalCharges to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# 7. Fill missing values
for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include='object').columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

# 8. Encode categorical columns
df = pd.get_dummies(df, drop_first=True, dtype=int)

# 9. Standardize numerical columns
cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
```

```python
scaler = StandardScaler()
df[cols] = scaler.fit_transform(df[cols])

# 10. Monthly Charges distribution
sns.histplot(df['MonthlyCharges'], kde=True)
plt.title("Monthly Charges Distribution")
plt.show()

# 11. Tenure distribution
sns.histplot(df['tenure'], kde=True)
plt.title("Tenure Distribution")
plt.show()

# 12. Correlation heatmap
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# 13. Save cleaned dataset
df.to_csv('Cleaned_Churn.csv', index=False)

print("Data preprocessing completed successfully!")
```

```
Dataset loaded successfully!
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
```

```
Dataset loaded successfully!
...
     customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

                PaymentMethod MonthlyCharges  TotalCharges Churn
0            Electronic check          29.85         29.85    No
1                Mailed check          56.95        1889.5    No
2                Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4            Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

```
Shape: (7043, 21)
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes
5  9305-CDSKC  Female              0      No         No       8          Yes
6  1452-KIOVK    Male              0      No        Yes      22          Yes
7  6713-OKOMC  Female              0      No         No      10           No
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No
5               Yes     Fiber optic             No  ...              Yes
6               Yes     Fiber optic             No  ...               No
7  No phone service             DSL            Yes  ...               No
8               Yes     Fiber optic             No  ...              Yes
9                No             DSL            Yes  ...               No

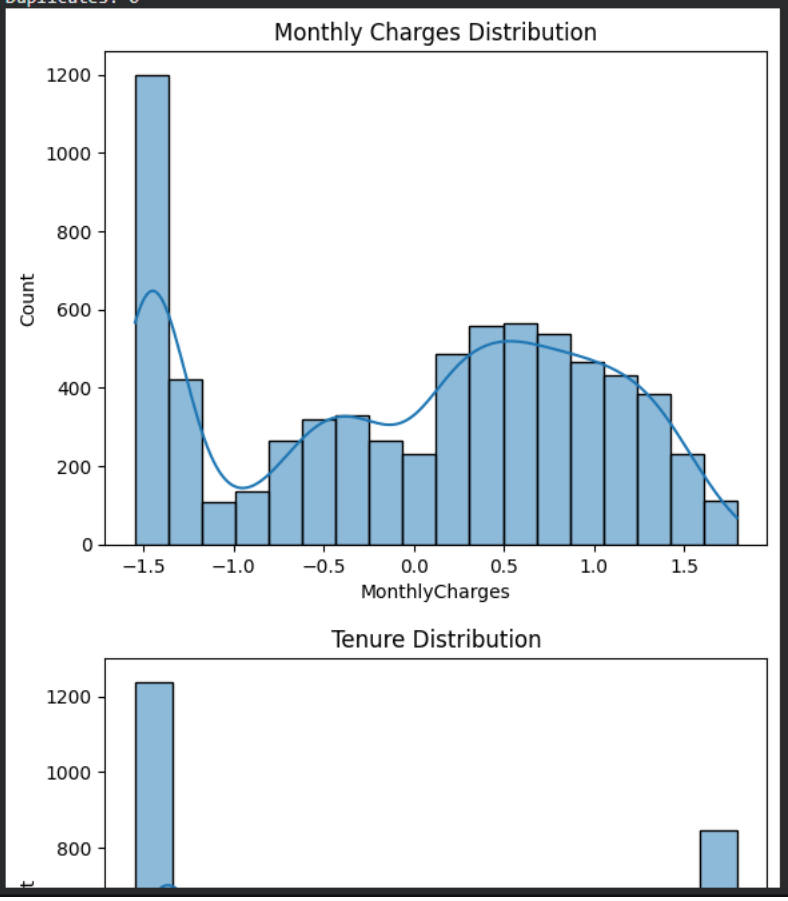
  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
```

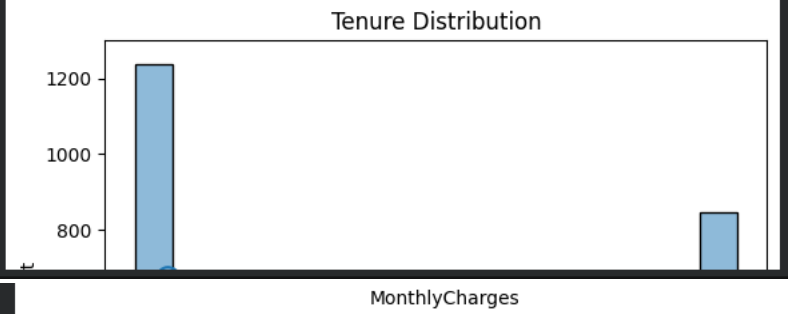
```
              PaymentMethod MonthlyCharges  TotalCharges Churn
0          Electronic check          29.85         29.85    No
1              Mailed check          56.95        1889.5    No
2              Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4          Electronic check          70.70        151.65   Yes
5          Electronic check          99.65         820.5   Yes
6    Credit card (automatic)          89.10        1949.4    No
7              Mailed check          29.75         301.9    No
8          Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

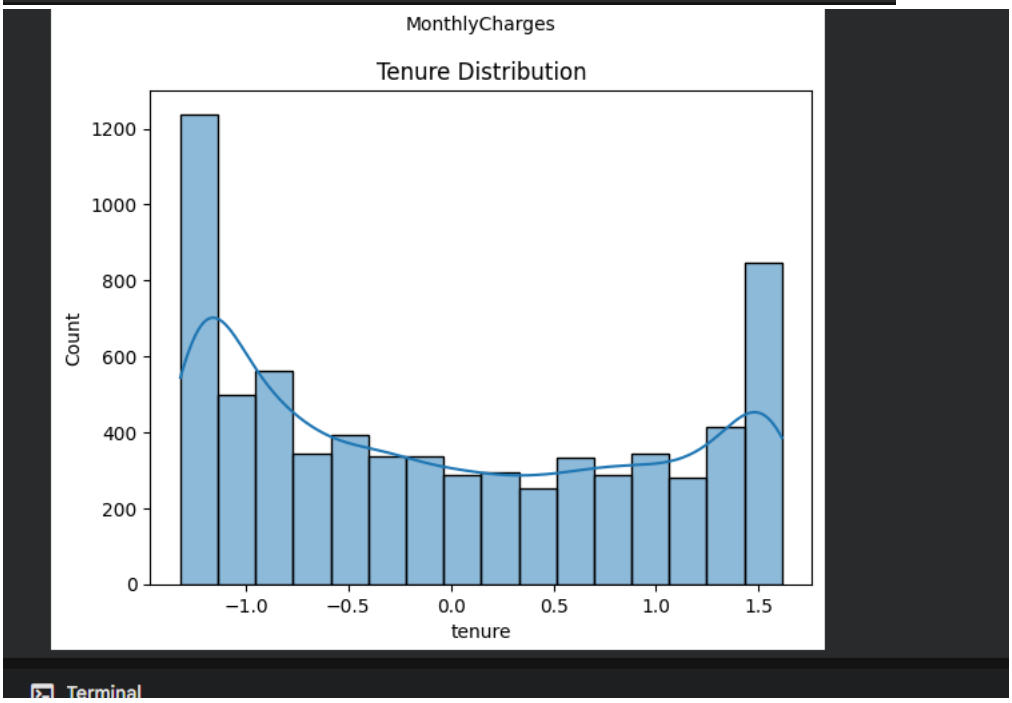
[10 rows x 21 columns]
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
7035  8456-QDAVC    Male              0      No         No      19
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
```

Duplicates: 0







Terminal

Experiment 2:

```python
# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 2. Load dataset automatically
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"

df = pd.read_csv(url)

# 3. Explore dataset
print("First 5 records:")
print(df.head())

print("\nLast 5 records:")
print(df.tail())

print("\nShape:", df.shape)
print("\nDataset Information:")
df.info()

print("\nSummary Statistics:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicates:", df.duplicated().sum())

# 4. Clean dataset
df.drop_duplicates(inplace=True)

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include='object').columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

# 5. Analyze customer churn
plt.figure(figsize=(5,4))
sns.countplot(data=df, x='Churn')
plt.title("Customer Churn Distribution")
plt.show()

# 6. Gender distribution
sns.countplot(data=df, x='gender')
plt.title("Gender Distribution")
plt.show()

# 7. Contract distribution
sns.countplot(data=df, x='Contract')
plt.title("Contract Type Distribution")
plt.show()

# 8. Payment method distribution
sns.countplot(data=df, x='PaymentMethod')
plt.xticks(rotation=45)
plt.title("Payment Method Distribution")
plt.show()

# 9. Histograms
df[['MonthlyCharges', 'TotalCharges', 'tenure']].hist(figsize=(10,6))
plt.show()
```

```
plt.show()

# 10. Boxplots
for col in ['MonthlyCharges', 'TotalCharges', 'tenure']:
    sns.boxplot(x=df[col])
    plt.title(col + " Boxplot")
    plt.show()

# 11. Detect outliers using IQR
for col in ['MonthlyCharges', 'TotalCharges', 'tenure']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    outliers = df[(df[col] < Q1 - 1.5*IQR) |
                  (df[col] > Q3 + 1.5*IQR)]

    print(col, "Outliers:", len(outliers))

# 12. Contract vs Churn
sns.countplot(data=df, x='Contract', hue='Churn')
plt.title("Contract vs Churn")
plt.show()

# 13. Internet Service vs Churn
sns.countplot(data=df, x='InternetService', hue='Churn')
plt.title("Internet Service vs Churn")
plt.show()

# 14. Monthly Charges vs Churn
sns.boxplot(data=df, x='Churn', y='MonthlyCharges')
plt.title("Monthly Charges vs Churn")
plt.show()

# 15. Tenure vs Churn
```

```
sns.boxplot(data=df, x='Churn', y='tenure')
plt.title("Tenure vs Churn")
plt.show()

# 16. Analytical questions
print("\nChurn by Contract:")
print(df.groupby('Contract')['Churn'].value_counts())

print("\nPayment Method used by Churned Customers:")
print(df[df['Churn'] == 'Yes']['PaymentMethod'].value_counts())

print("\nAverage Tenure by Churn:")
print(df.groupby('Churn')['tenure'].mean())

print("\nAverage Monthly Charges by Churn:")
print(df.groupby('Churn')['MonthlyCharges'].mean())

print("\nInternet Service Churn:")
print(df[df['Churn'] == 'Yes']['InternetService'].value_counts())

# 17. Encode categorical columns
df = pd.get_dummies(df, drop_first=True, dtype=int)

# 18. Split dataset into training and testing
X = df.drop('Churn_Yes', axis=1)
y = df['Churn_Yes']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# 19. Save processed dataset
```

```
# 19. Save processed dataset
df.to_csv('CustomerChurn_Preprocessed.csv', index=False)

print("\nExperiment 2 Completed Successfully!")
```

```
First 5 records:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]

Last 5 records:
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
```

```
print("\nExperiment 2 Completed Successfully!")
```

```
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService OnlineSecurity  ...  \
7038          Yes               Yes             DSL            Yes  ...
7039          Yes               Yes     Fiber optic             No  ...
7040           No  No phone service             DSL            Yes  ...
7041          Yes               Yes     Fiber optic             No  ...
7042          Yes                No     Fiber optic            Yes  ...

     DeviceProtection TechSupport StreamingTV StreamingMovies        Contract  \
7038              Yes         Yes         Yes             Yes        One year
7039              Yes          No         Yes             Yes        One year
7040               No          No          No              No  Month-to-month
7041               No          No          No              No  Month-to-month
7042              Yes         Yes         Yes             Yes        Two year

     PaperlessBilling              PaymentMethod MonthlyCharges  TotalCharges  \
7038              Yes               Mailed check          84.80        1990.5
7039              Yes    Credit card (automatic)         103.20        7362.9
7040              Yes           Electronic check          29.60        346.45
7041              Yes               Mailed check          74.40         306.6
7042              Yes  Bank transfer (automatic)         105.65        6844.5

     Churn
7038    No
7039    No
7040    No
7041   Yes
7042    No
```

```
print("\nExperiment 2 Completed Successfully!")
```

```
[5 rows x 21 columns]

Shape: (7043, 21)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
print("\nExperiment 2 Completed Successfully!")
```

```
std          0.368612    24.559481       30.090047
min          0.000000     0.000000       18.250000
25%          0.000000     9.000000       35.500000
50%          0.000000    29.000000       70.350000
75%          0.000000    55.000000       89.850000
max          1.000000    72.000000      118.750000

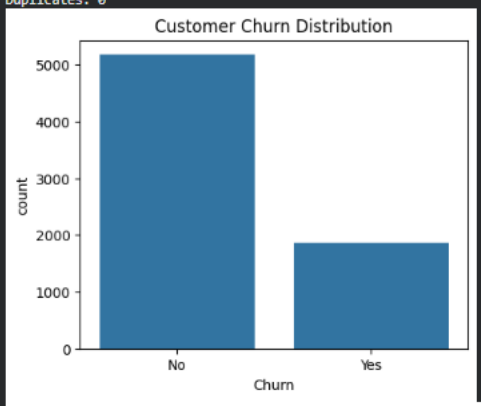
Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

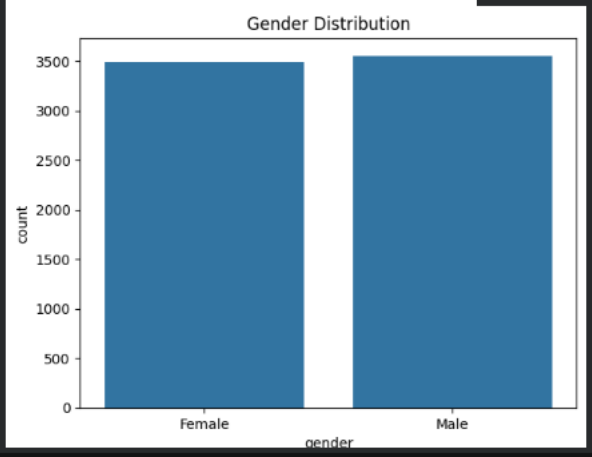
Duplicates: 0
```

Customer Churn Distribution

```
print("\nExperiment 2 Completed Successfully!")
```

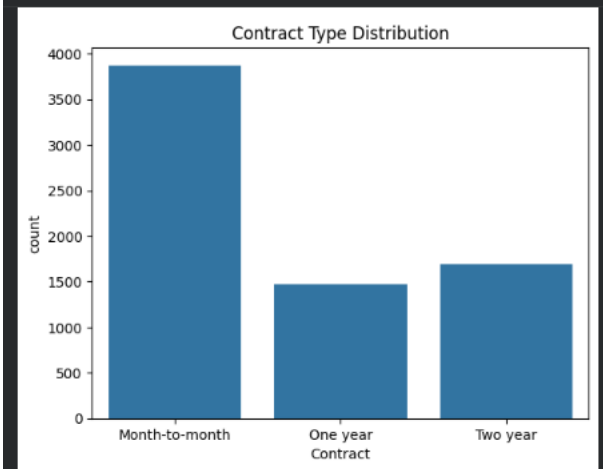
```
Duplicates: 0
```

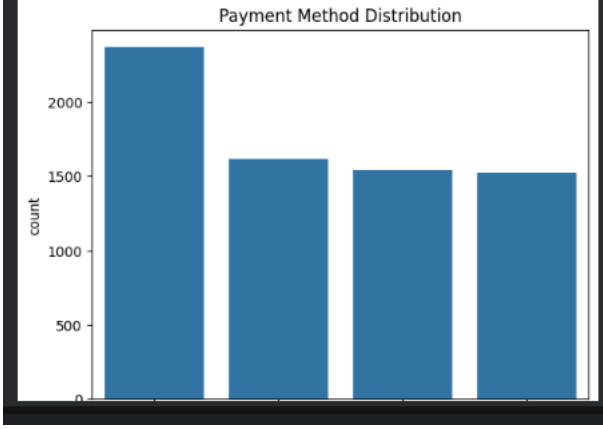


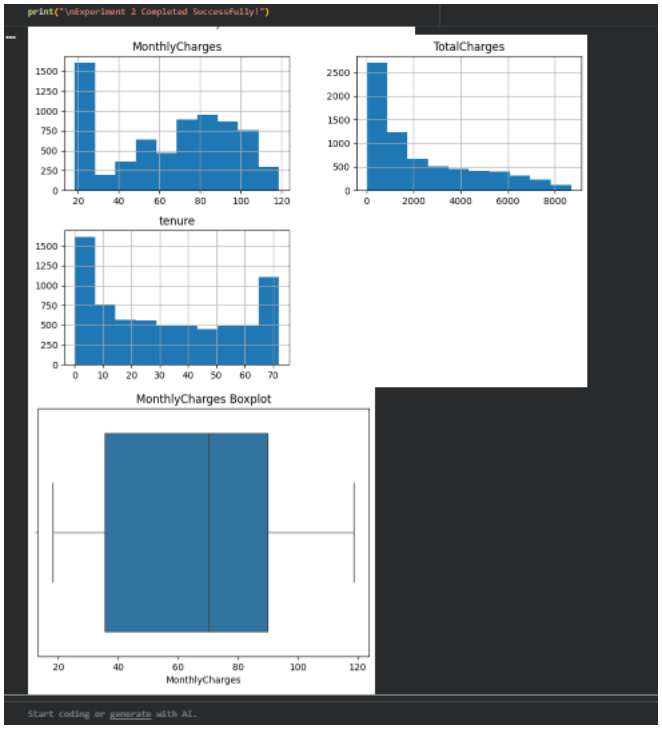


riables Terminal

```
print("\nExperiment 2 Completed Successfully!")
```







Experiment 3(a): Classification

ATTANA S

241801030 Experiment 3(a): Classification

+ Code + Text

```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# 2. Load dataset
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(url)

# 3. Preprocessing
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

df = pd.get_dummies(df, drop_first=True, dtype=int)

# 4. Split data
```

Terminal

```python
X = df.drop('Churn_Yes', axis=1)
y = df['Churn_Yes']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 5. Create models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42)
}

# 6. Train and evaluate models
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    results.append([
        name,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred),
        recall_score(y_test, pred),
        f1_score(y_test, pred)
    ])

    print("\n", name)
```

Terminal

[1e] ✓ 1m

```python
    print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

# 7. Display results
result_df = pd.DataFrame(
    results,
    columns=['Model', 'Accuracy', 'Precision', 'Recall', 'F1 Score']
)

print(result_df)

# 8. Compare models
result_df.set_index('Model').plot(kind='bar', figsize=(8,5))
plt.title("Classification Model Comparison")
plt.ylabel("Score")
plt.show()

# 9. Best model
best = result_df.loc[result_df['F1 Score'].idxmax()]
print("\nBest Model:", best['Model'])

# 10. Recommendations
print("\nRecommendations:")
print("1. Give offers to customers with short tenure.")
print("2. Provide better contract plans.")
print("3. Improve customer service.")

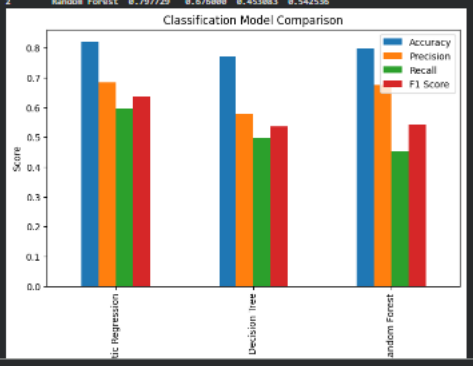
print("\nExperiment 3(a) Completed!")
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to co
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

```
Logistic Regression
Confusion Matrix:
 [[933 103]
 [150 223]]

Decision Tree
Confusion Matrix:
 [[901 135]
 [187 186]]

Random Forest
Confusion Matrix:
 [[955  81]
 [204 169]]
                 Model  Accuracy  Precision    Recall  F1 Score
0  Logistic Regression  0.820440   0.684049  0.597855  0.638054
1        Decision Tree  0.771469   0.579439  0.498660  0.536023
2        Random Forest  0.797729   0.676000  0.453083  0.542536
```



EXPERMINET 3B:

ATTANA S

241801030 Experiment 3(b): Regression

```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 2. Load dataset
url = "https://raw.githubusercontent.com/selva86/datasets/master/Advertising.csv"
df = pd.read_csv(url)

# 3. Explore dataset
print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.isnull().sum())
print(df.describe())

# 4. Select features and target
# 4. Select features and target
print(df.columns)

X = df[['TV', 'radio', 'newspaper']]
y = df['sales']
```

0s

```python
# 5. Correlation heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 7. Create models
models = {
    "Linear Regression": LinearRegression(),
    "Polynomial Regression": make_pipeline(
        PolynomialFeatures(2), LinearRegression()
    ),
    "SVR": make_pipeline(StandardScaler(), SVR(kernel='rbf'))
}

# 8. Train and evaluate
results = []
predictions = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    predictions[name] = pred

    results.append([
        name,
        mean_absolute_error(y_test, pred),
        mean_squared_error(y_test, pred),
        np.sqrt(mean_squared_error(y_test, pred)),
        r2_score(y_test, pred)
    ])
```

```
# 9. Display results
result_df = pd.DataFrame(
    results,
    columns=['Model', 'MAE', 'MSE', 'RMSE', 'R2 Score']
)

print(result_df)

# 10. Compare models
result_df.set_index('Model').plot(kind='bar', figsize=(9,5))
plt.title("Regression Model Comparison")
plt.show()

# 11. Best model
best = result_df.loc[result_df['RMSE'].idxmin()]
print("Best Model:", best['Model'])

# 12. Actual vs Predicted
best_pred = predictions[best['Model']]

plt.scatter(y_test, best_pred)
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")
plt.show()

# 13. Recommendations
print("1. Invest more in effective advertising channels.")
print("2. Monitor advertising performance regularly.")
print("3. Optimize the advertising budget using predictions.")

print("Experiment 3(b) Completed Successfully!")
```

```
   Unnamed: 0     TV  radio  newspaper  sales
0           1  230.1   37.8       69.2   22.1
```

bles Terminal

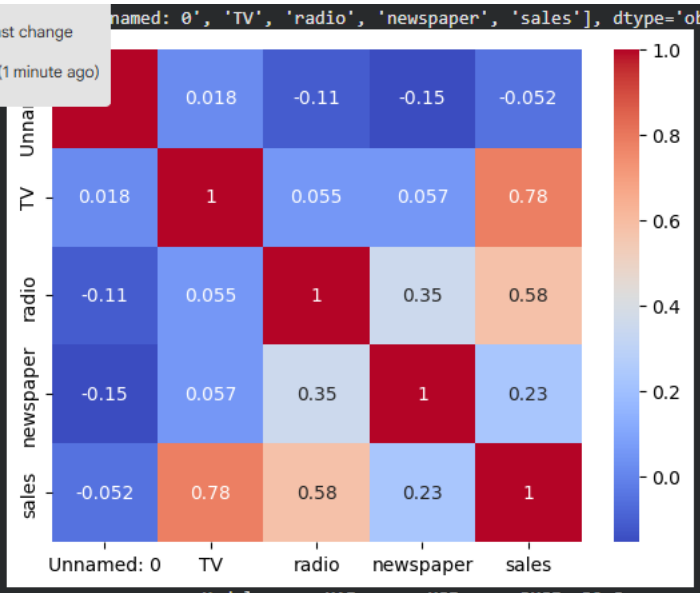
```
   Unnamed: 0     TV  radio  newspaper  sales
0           1  230.1   37.8       69.2   22.1
1           2   44.5   39.3       45.1   10.4
2           3   17.2   45.9       69.3    9.3
3           4  151.5   41.3       58.5   18.5
4           5  180.8   10.8       58.4   12.9
     Unnamed: 0     TV  radio  newspaper  sales
195         196   38.2    3.7       13.8    7.6
196         197   94.2    4.9        8.1    9.7
197         198  177.0    9.3        6.4   12.8
198         199  283.6   42.0       66.2   25.5
199         200  232.1    8.6        8.7   13.4
Shape: (200, 5)
Unnamed: 0    0
TV            0
radio         0
newspaper     0
sales         0
dtype: int64
       Unnamed: 0          TV       radio   newspaper       sales
count  200.000000  200.000000  200.000000  200.000000  200.000000
mean   100.500000  147.042500   23.264000   30.554000   14.022500
std     57.879185   85.854236   14.846809   21.778621    5.217457
min      1.000000    0.700000    0.000000    0.300000    1.600000
25%     50.750000   74.375000    9.975000   12.750000   10.375000
50%    100.500000  149.750000   22.900000   25.750000   12.900000
75%    150.250000  218.825000   36.525000   45.100000   17.400000
max    200.000000  296.400000   49.600000  114.000000   27.000000
Index(['Unnamed: 0', 'TV', 'radio', 'newspaper', 'sales'], dtype='object')
```

0 1.0

un cell (Ctrl+Enter)
ell executed since last change

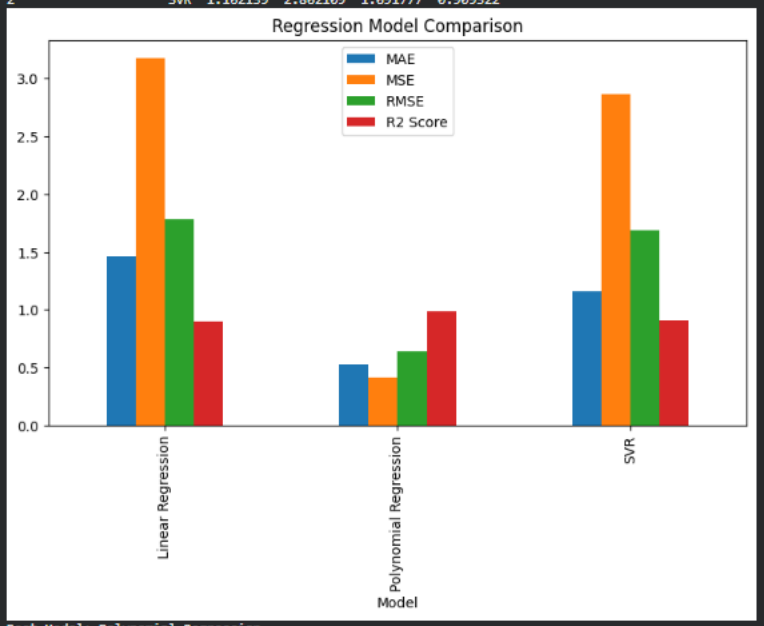
xecuted at 6:43 PM (1 minute ago)
xecuted in 0.722s

```
...named: 0', 'TV', 'radio', 'newspaper', 'sales'], dtype='object')
```



	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	1.460757	3.174097	1.781600	0.899438
1	Polynomial Regression	0.526179	0.412910	0.642581	0.986918
2	SVR	1.162139	2.862109	1.691777	0.909322

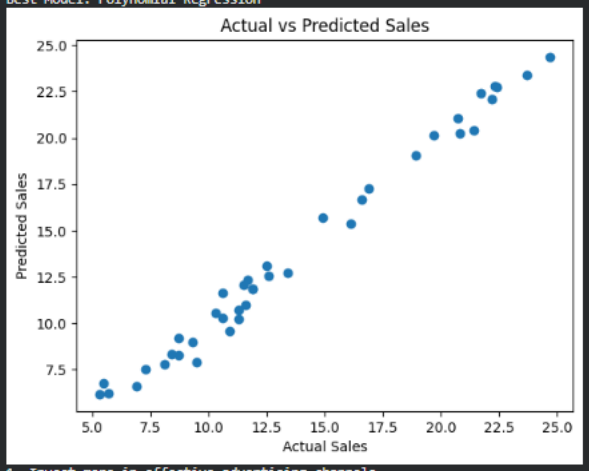
	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	1.460757	3.174097	1.781600	0.899438
1	Polynomial Regression	0.526179	0.412910	0.642581	0.986918
2	SVR	1.162139	2.862109	1.691777	0.909322




```
Best Model: Polynomial Regression
```

Model

Best Model: Polynomial Regression



1. Invest more in effective advertising channels.
2. Monitor advertising performance regularly.
3. Optimize the advertising budget using predictions.
Experiment 3(b) Completed Successfully!

Experminet 5:

```python
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Load dataset (Updated to a working, active URL containing the same dataset)
# Load Netflix dataset automatically
import pandas as pd

url = "https://huggingface.co/datasets/hugginglearners/netflix-shows/resolve/main/netflix_titles.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print(df.head())
print("Shape:", df.shape)

# 3. Explore dataset
print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.info())
print(df.isnull().sum())
print(df.describe())

# 4. Movies vs TV Shows
sns.countplot(data=df, x='type')
plt.title("Movies vs TV Shows")
plt.show()

# 5. Ratings distribution
plt.figure(figsize=(10,5))
sns.countplot(data=df, x='rating',
              order=df['rating'].value_counts().index)
```

```
plt.xticks(rotation=90)
plt.title("Netflix Ratings")
plt.show()

# 6. Top 10 countries
countries = df['country'].dropna().str.split(', ').explode()
countries.value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Countries")
plt.show()

# 7. Release year
df['release_year'].value_counts().sort_index().plot(figsize=(10,4))
plt.title("Netflix Release Year Trend")
plt.show()

# 8. Content added over years
df['date_added'] = pd.to_datetime(df['date_added'].str.strip())
df['date_added'].dt.year.value_counts().sort_index().plot(kind='bar')
plt.title("Content Added to Netflix")
plt.show()

# 9. Movie duration
movies = df[df['type']=='Movie'].copy()
movies['duration_num'] = pd.to_numeric(
    movies['duration'].str.extract(r'(\d+)')[0]
)
sns.histplot(movies['duration_num'].dropna(), bins=20)
plt.title("Movie Duration Distribution")
plt.show()

# 10. TV Show seasons
tv = df[df['type']=='TV Show']
tv['duration'].value_counts().head(10).plot(kind='bar')
plt.title("TV Show Seasons")
plt.show()
```

```python
# 11. Top 10 genres
genres = df['listed_in'].dropna().str.split(', ').explode()
genres.value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Netflix Genres")
plt.show()

# 12. Top directors
directors = df['director'].dropna().str.split(', ').explode()
directors.value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Directors")
plt.show()

# 13. Top actors
actors = df['cast'].dropna().str.split(', ').explode()
actors.value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Actors")
plt.show()

# 14. Content added by month
df['date_added'].dt.month.value_counts().sort_index().plot(kind='bar')
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.show()

# 15. Oldest and newest titles
print("Oldest Title:")
print(df.loc[df['release_year'].idxmin(), ['title','release_year']])

print("Newest Title:")
print(df.loc[df['release_year'].idxmax(), ['title','release_year']])

# 16. Movies vs TV Shows by country
df['first_country'] = df['country'].str.split(', ').str[0]
sns.countplot(data=df, x='first_country', hue='type',
```

```python
# 16. Movies vs TV Shows by country
df['first_country'] = df['country'].str.split(', ').str[0]
sns.countplot(data=df, x='first_country', hue='type',
              order=df['first_country'].value_counts().head(10).index)
plt.xticks(rotation=90)
plt.title("Movies vs TV Shows by Country")
plt.show()

# 17. Ratings by content type
sns.countplot(data=df, x='rating', hue='type',
              order=df['rating'].value_counts().head(10).index)
plt.xticks(rotation=90)
plt.title("Ratings by Content Type")
plt.show()

# 18. Save dataset
df.to_csv('Netflix_Analysis.csv', index=False)

print("Experiment 5 Completed Successfully!")
```

```
Dataset loaded successfully!
  show_id     type                  title         director  \
0      s1    Movie   Dick Johnson Is Dead  Kirsten Johnson
1      s2  TV Show          Blood & Water              NaN
2      s3  TV Show              Ganglands  Julien Leclercq
3      s4  TV Show  Jailbirds New Orleans              NaN
4      s5  TV Show           Kota Factory              NaN

                                          cast        country  \
0                                          NaN  United States
1  Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...   South Africa
2  Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...            NaN
3                                          NaN            NaN
4  Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...          India

           date_added  release_year rating   duration  \
0  September 25, 2021          2020  PG-13     90 min
```

```
             date_added  release_year rating   duration  \
0  September 25, 2021          2020  PG-13     90 min
1  September 24, 2021          2021  TV-MA  2 Seasons
2  September 24, 2021          2021  TV-MA   1 Season
3  September 24, 2021          2021  TV-MA   1 Season
4  September 24, 2021          2021  TV-MA  2 Seasons

                                      listed_in  \
0                                  Documentaries
1    International TV Shows, TV Dramas, TV Mysteries
2  Crime TV Shows, International TV Shows, TV Act...
3                            Docuseries, Reality TV
4  International TV Shows, Romantic TV Shows, TV ...

                                     description
0  As her father nears the end of his life, filmm...
1  After crossing paths at a party, a Cape Town t...
2  To protect his family from a powerful drug lor...
3  Feuds, flirtations and toilet talk go down amo...
4  In a city of coaching centers known to train I...
Shape: (8807, 12)
  show_id     type                  title         director  \
0      s1    Movie   Dick Johnson Is Dead  Kirsten Johnson
1      s2  TV Show          Blood & Water              NaN
2      s3  TV Show              Ganglands  Julien Leclercq
3      s4  TV Show  Jailbirds New Orleans              NaN
4      s5  TV Show           Kota Factory              NaN

                                          cast        country  \
0                                          NaN  United States
1  Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...   South Africa
2  Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...            NaN
3                                          NaN            NaN
4  Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...          India

             date_added  release_year rating   duration  \
0  September 25, 2021          2020  PG-13     90 min
1  September 24, 2021          2021  TV-MA  2 Seasons
3  September 24, 2021          2021  TV-MA   1 Season
4  September 24, 2021          2021  TV-MA  2 Seasons

                                      listed_in  \
0                                  Documentaries
1    International TV Shows, TV Dramas, TV Mysteries
2  Crime TV Shows, International TV Shows, TV Act...
3                            Docuseries, Reality TV
4  International TV Shows, Romantic TV Shows, TV ...

                                     description
0  As her father nears the end of his life, filmm...
1  After crossing paths at a party, a Cape Town t...
2  To protect his family from a powerful drug lor...
3  Feuds, flirtations and toilet talk go down amo...
4  In a city of coaching centers known to train I...
     show_id     type        title         director  \
8802   s8803    Movie       Zodiac    David Fincher
8803   s8804  TV Show  Zombie Dumb              NaN
8804   s8805    Movie   Zombieland  Ruben Fleischer
8805   s8806    Movie         Zoom     Peter Hewitt
8806   s8807    Movie       Zubaan      Mozez Singh

                                          cast        country  \
8802  Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...  United States
8803                                                NaN            NaN
8804  Jesse Eisenberg, Woody Harrelson, Emma Stone, ...  United States
8805  Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...  United States
8806  Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...          India

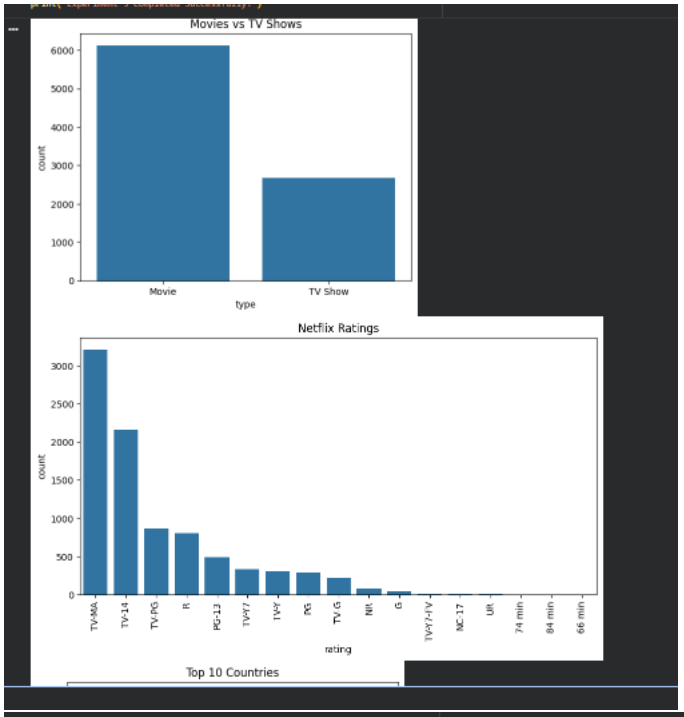
             date_added  release_year rating   duration  \
8802  November 20, 2019          2007      R    158 min
8803       July 1, 2019          2018  TV-Y7  2 Seasons
8804   November 1, 2019          2009      R     88 min
8805   January 11, 2020          2006     PG     88 min
8806      March 2, 2019          2015  TV-14    111 min

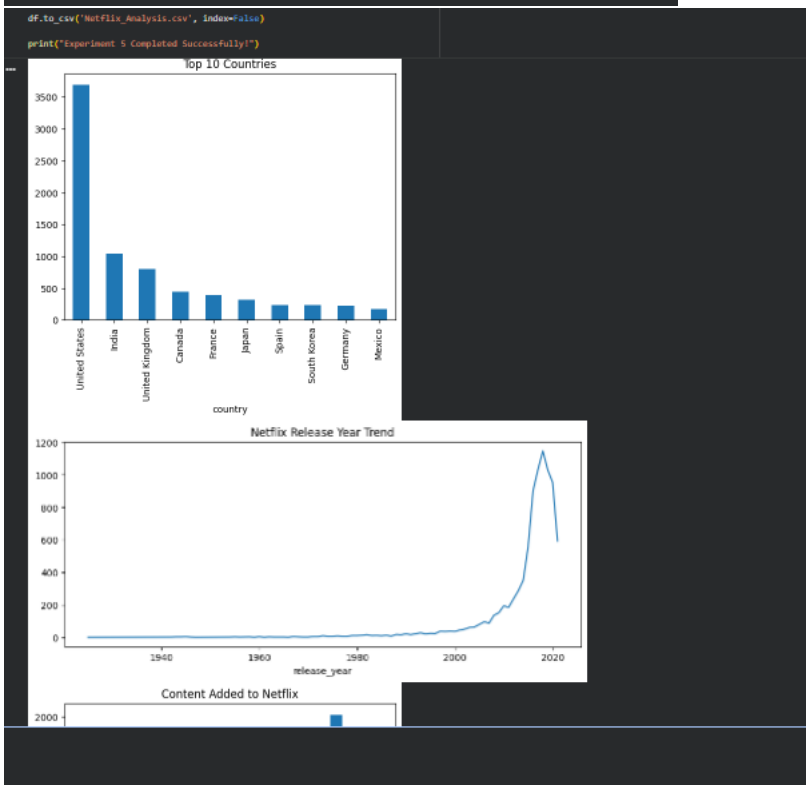
                                      listed_in  \
```

```
8802                  Cult Movies, Dramas, Thrillers
8803          Kids' TV, Korean TV Shows, TV Comedies
8804                          Comedies, Horror Movies
8805              Children & Family Movies, Comedies
8806  Dramas, International Movies, Music & Musicals

                                              description
8802  A political cartoonist, a crime reporter and a...
8803  While living alone in a spooky town, a young g...
8804  Looking to survive in a world taken over by zo...
8805  Dragged from civilian life, a former superhero...
8806  A scrappy but poor boy worms his way into a ty...
Shape: (8807, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
None
show_id            0
type               0
title              0
director        2634
cast             825
```

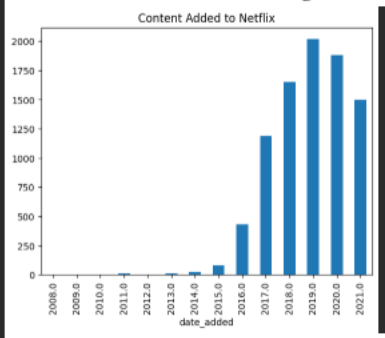
```
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
None
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
       release_year
count   8807.000000
mean    2014.180198
std        8.819312
min     1925.000000
25%     2013.000000
50%     2017.000000
75%     2019.000000
max     2021.000000
```

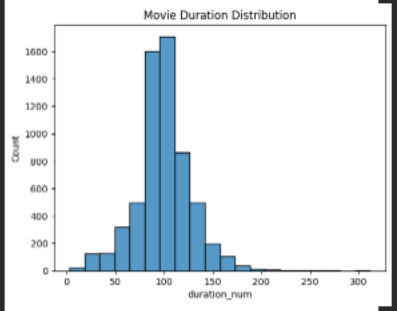

Movies vs TV Shows
count
type
Movie
TV Show
Netflix Ratings
count
rating
TV-MA
TV-14
TV-PG
R
PG-13
TV-Y7
TV-Y
PG
TV-G
NR
G
TV-Y7-FV
NC-17
UR
74 min
84 min
66 min
Top 10 Countries


df.to_csv('Netflix_Analysis.csv', index=False)
print("Experiment 5 Completed Successfully!")
Top 10 Countries
United States
India
United Kingdom
Canada
France
Japan
Spain
South Korea
Germany
Mexico
country
Netflix Release Year Trend
release_year
Content Added to Netflix

```
print("Experiment 5 Completed Successfully!")
```

release_year

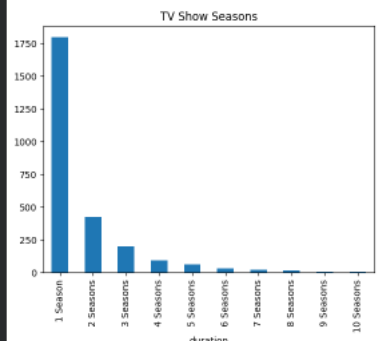


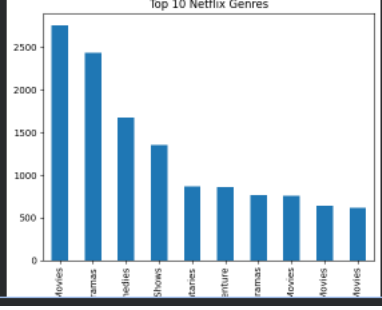


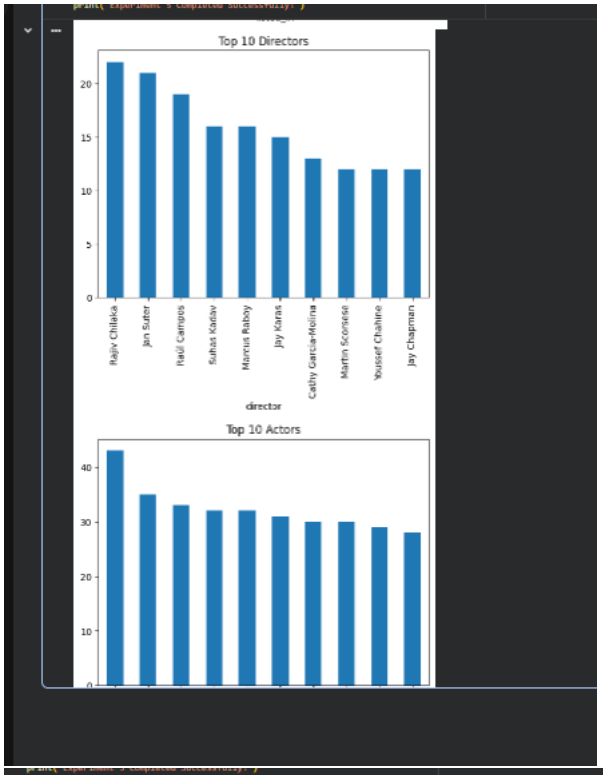
TV Show Seasons

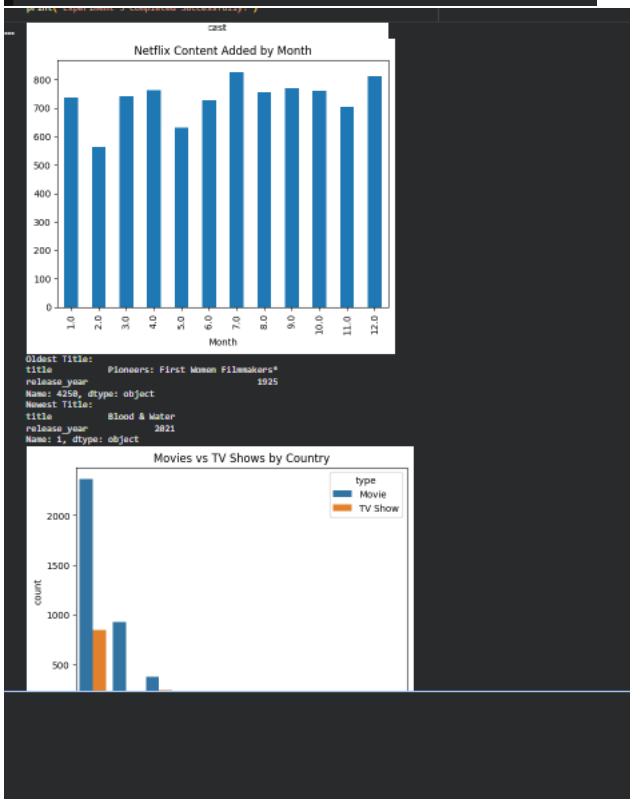
```
print("Experiment 5 Completed Successfully!")
```

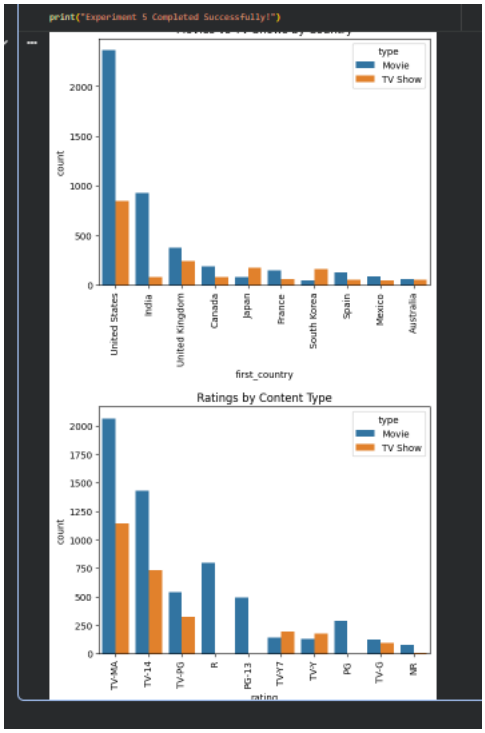
duration_num






Top 10 Directors
Rajiv Chilaka
Jan Suter
Raúl Campos
Suhas Kadav
Marcus Raboy
Jay Karas
Cathy Garcia-Molina
Martin Scorsese
Youssef Chahine
Jay Chapman
director
Top 10 Actors


cast
Netflix Content Added by Month
Month
Oldest Title:
title Pioneers: First Women Filmmakers*
release_year 1925
Name: 4250, dtype: object
Newest Title:
title Blood & Water
release_year 2021
Name: 1, dtype: object
Movies vs TV Shows by Country
type
Movie
TV Show
count



Experiment 6:

ATTANA S

24180130 Experiment 6

[18] ✓ 16s

```
# Install required library
!pip install -q statsmodels

# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# 1. Load dataset automatically
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url)

df['Month'] = pd.to_datetime(df['Month'])
df.set_index('Month', inplace=True)
df.index.freq = 'MS'

print(df.head())

# 2. Plot time series
plt.figure(figsize=(8,4))
plt.plot(df['Passengers'])
plt.title("Monthly Airline Passengers")
```

```
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.show()

# 3. ADF stationarity test
result = adfuller(df['Passengers'])
print("ADF p-value:", result[1])

# 4. Differencing
df['Difference'] = df['Passengers'].diff()

plt.figure(figsize=(8,4))
plt.plot(df['Difference'])
plt.title("Differenced Data")
plt.show()

# 5. Split data (80% training, 20% testing)
train_size = int(len(df) * 0.8)
train = df['Passengers'][:train_size]
test = df['Passengers'][train_size:]

# 6. Build ARIMA model
model = ARIMA(train, order=(2,1,2))
model_fit = model.fit()

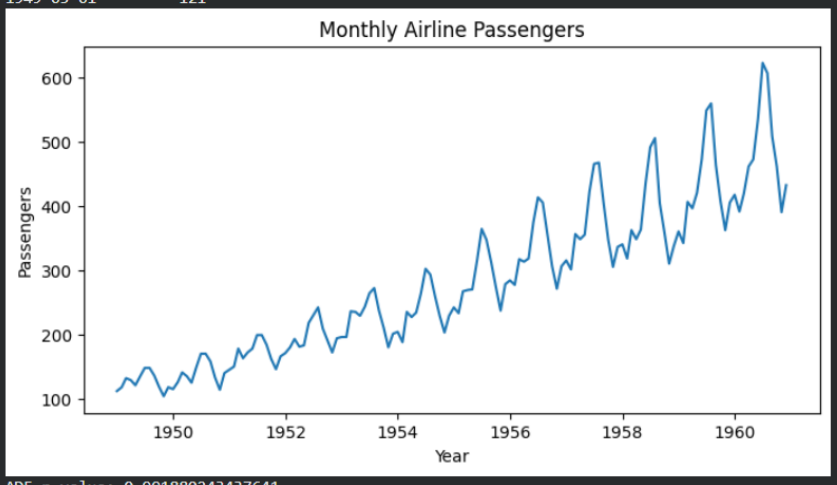
# 7. Forecast
predictions = model_fit.forecast(steps=len(test))

# 8. Evaluate model
print("MAE:", mean_absolute_error(test, predictions))
```

```python
# 9. Actual vs Predicted graph
plt.figure(figsize=(9,4))
plt.plot(test.index, test, label="Actual")
plt.plot(test.index, predictions, label="Predicted")
plt.title("ARIMA Forecast")
plt.xlabel("Date")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```

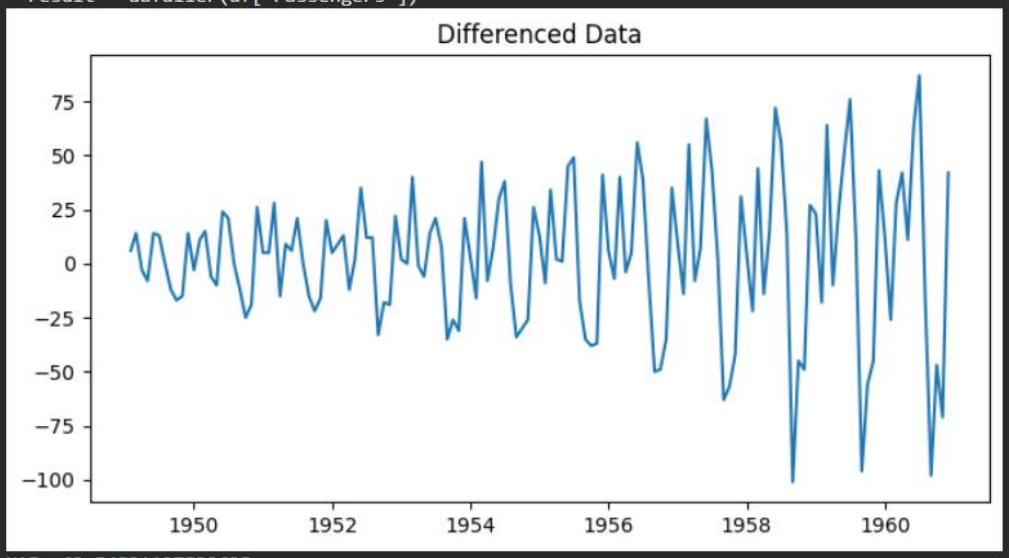
```
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```

```
1949-05-01         121
```




```
ADF p-value: 0.991880243437641
/tmp/ipykernel_10944/3875719875.py:31: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store
  result = adfuller(df['Passengers'])
```

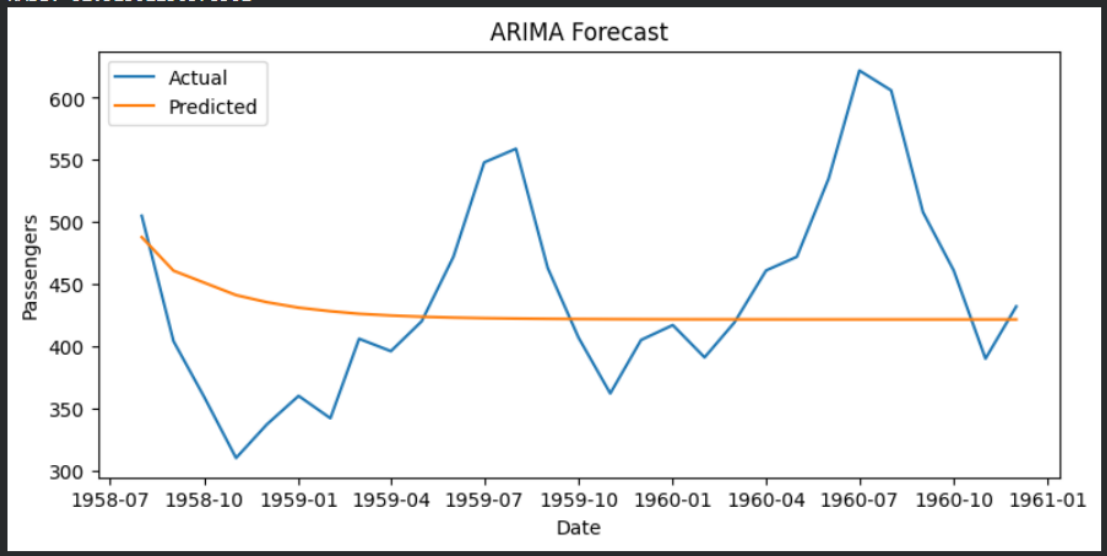
```
/tmp/ipykernel_10944/3875719875.py:31: FutureWarning: adfuller currently returns a plain tuple whose l
  result = adfuller(df['Passengers'])
```




```
MAE: 63.54531127532635
MSE: 6808.397034418323
RMSE: 82.51301130378361
```

1950 1952 1954 1956 1958 1960

MAE: 63.54531127532635
MSE: 6808.397034418323
RMSE: 82.51301130378361



Experiment 7:

```python
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Load dataset automatically
url = "https://raw.githubusercontent.com/MahimaSriK/Mall-Customer-Segmentation-Dataset/main/Mall_Customers.csv"
df = pd.read_csv(url)

print(df.head())

# 2. Select features
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# 3. Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Elbow method
wcss = []

for i in range(1, 11):
    model = KMeans(n_clusters=i, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
```

```python
# 5. Apply K-Means (5 clusters)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# 6. Plot clusters and centroids
plt.figure(figsize=(8,5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
            c=df['Cluster'], cmap='rainbow')

plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color='black', marker='X', s=150,
            label='Centroids')

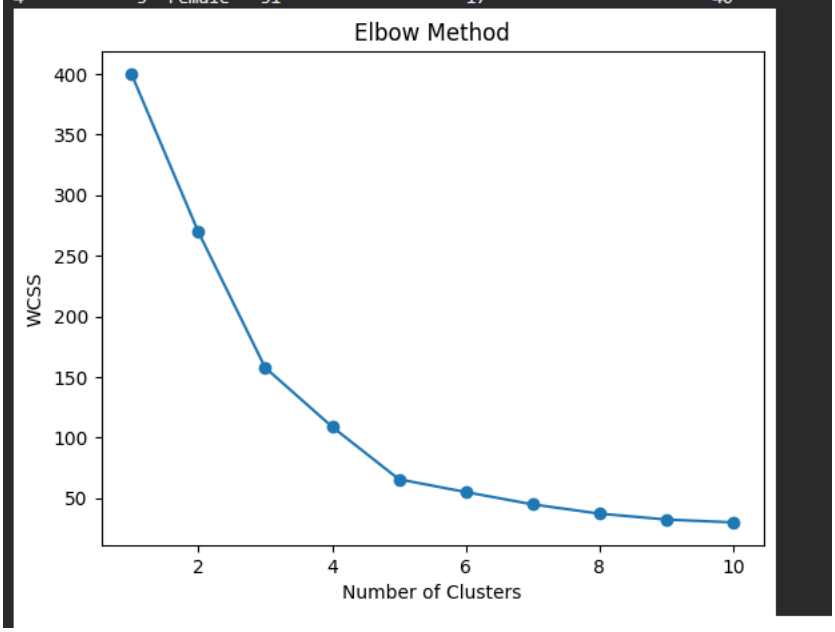
plt.title("Customer Segmentation")
plt.xlabel("Annual Income (Scaled)")
plt.ylabel("Spending Score (Scaled)")
plt.legend()
plt.show()

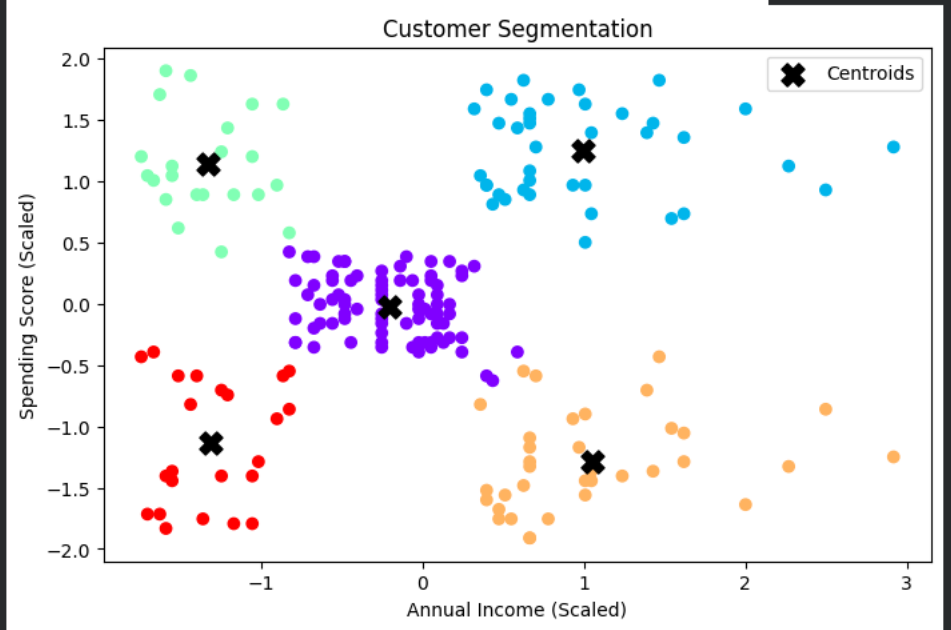
# 7. Cluster summary
print(df.groupby('Cluster')[[
    'Annual Income (k$)', 'Spending Score (1-100)'
]].mean())

# 8. Save dataset
df.to_csv("Mall_Customers_Clustered.csv", index=False)

print("Clustered dataset saved successfully!")
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40





Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

Clustered dataset saved successfully!

EXPERIMENT 8:

ATTANA S

24180130 Experiment 8:

```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 2. Load dataset
# 2. Load dataset
url = "https://raw.githubusercontent.com/hannazan/mall-customer-segmentation/main/data/Mall_Customers.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print(df.head())
print("Shape:", df.shape)

# 3. Explore dataset
print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.describe())

# 4. Select features
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# 5. Standardize features
scaler = StandardScaler()
X scaled = scaler.fit transform(X)
```

Terminal

```
X_scaled = scaler.fit_transform(X)

# 6. Scatter plot
plt.scatter(X.iloc[:, 0], X.iloc[:, 1])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Data")
plt.show()

# 7. Pair plot
import seaborn as sns
sns.pairplot(X)
plt.show()

# 8. Elbow method
wcss = []
for i in range(1, 11):
    model = KMeans(n_clusters=i, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

# 9. Apply K-Means with 5 clusters
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# 10. Cluster visualization
plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
            c=df['Cluster'], cmap='rainbow')
```

```
# 10. Cluster visualization
plt.scatter(X_scaled[:, 0], X_scaled[:, 1],
            c=df['Cluster'], cmap='rainbow')

plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color='black', marker='X', s=150)

plt.title("Customer Segmentation")
plt.xlabel("Annual Income (Scaled)")
plt.ylabel("Spending Score (Scaled)")
plt.show()

# 11. Cluster count and average
print("Customers in each cluster:")
print(df['Cluster'].value_counts())

print("\nCluster averages:")
print(df.groupby('Cluster')[[
    'Annual Income (k$)', 'Spending Score (1-100)'
]].mean())

# 12. Display centroids
print("Cluster Centroids:")
print(kmeans.cluster_centers_)

# 13. Save dataset
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("Dataset saved successfully!")
```

```
Dataset loaded successfully!
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
```

Output:

```
print("Dataset saved successfully!")
```

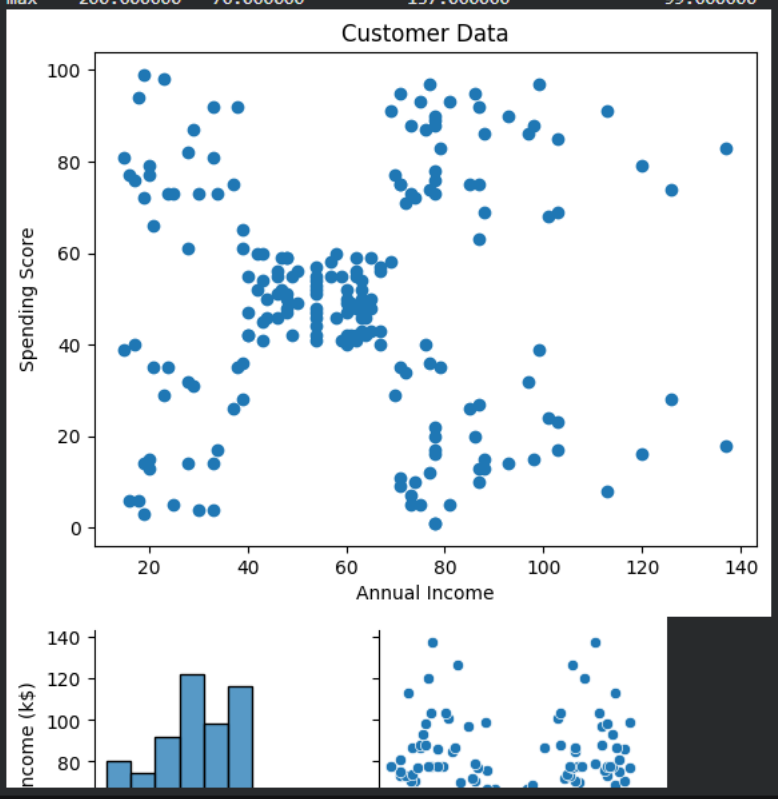
```
Dataset loaded successfully!
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
Shape: (200, 5)
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83
Shape: (200, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
```

Terminal

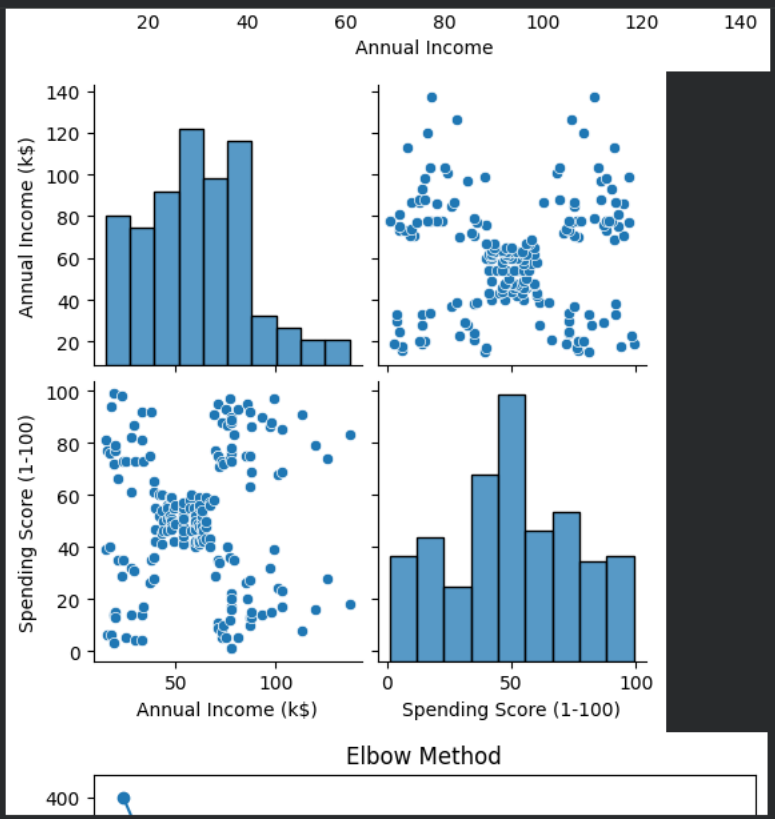
```
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
0
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000
```



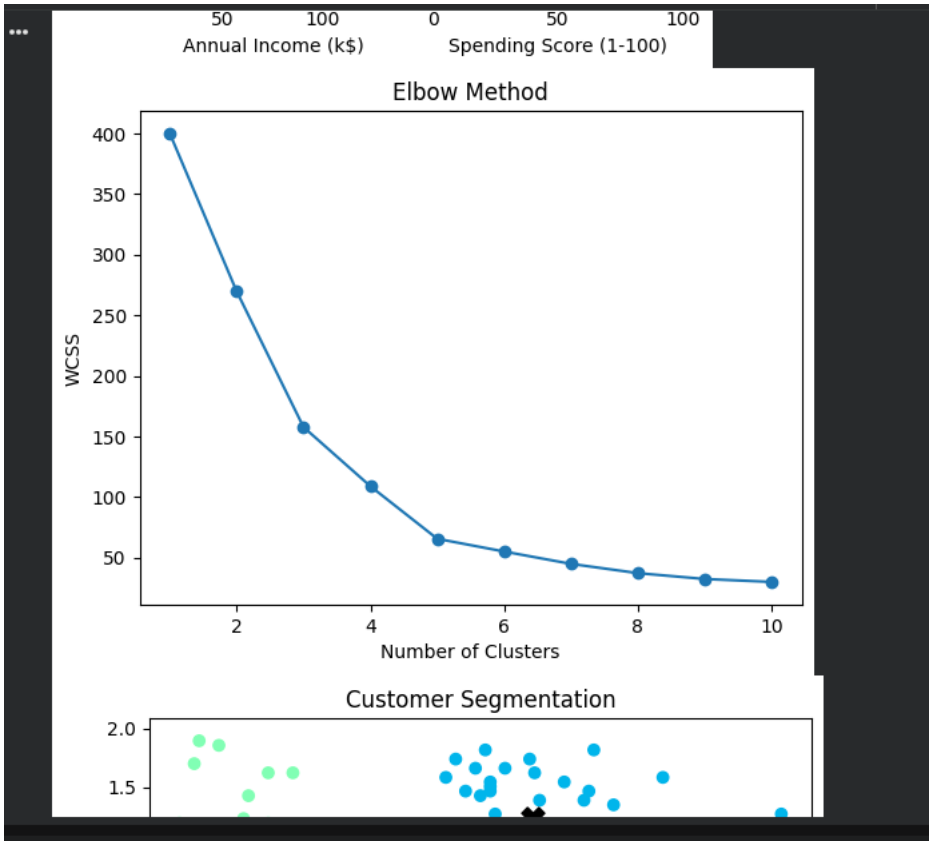
```
max    200.000000   70.000000          137.000000              99.000000
```

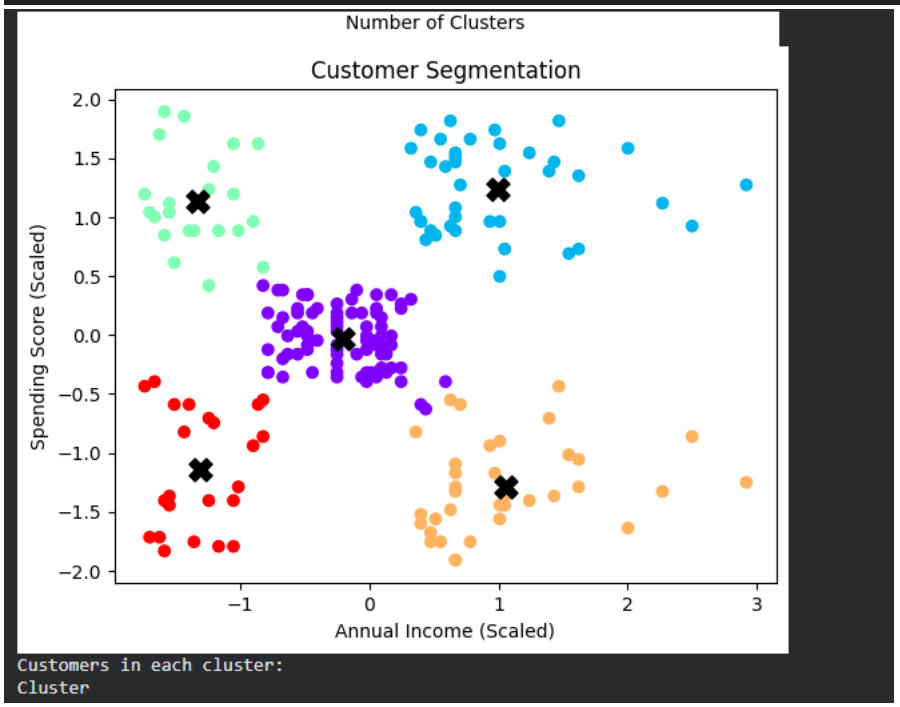


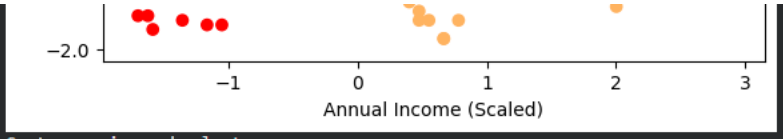

```
print( Dataset saved successfully! )
```



Terminal


50
100
0
50
100
Annual Income (k$)
Spending Score (1-100)
Elbow Method
WCSS
400
350
300
250
200
150
100
50
2
4
6
8
10
Number of Clusters
Customer Segmentation
2.0
1.5


Number of Clusters
Customer Segmentation
Spending Score (Scaled)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−1
0
1
2
3
Annual Income (Scaled)
Customers in each cluster:
Cluster




```
Customers in each cluster:
Cluster
0    81
1    39
3    35
4    23
2    22
Name: count, dtype: int64

Cluster averages:
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                 55.296296               49.518519
1                 86.538462               82.128205
2                 25.727273               79.363636
3                 88.200000               17.114286
4                 26.304348               20.913043
Cluster Centroids:
[[-0.20091257 -0.02645617]
 [ 0.99158305  1.23950275]
 [-1.32954532  1.13217788]
 [ 1.05500302 -1.28443907]
 [-1.30751869 -1.13696536]]
Dataset saved successfully!
```